

Донорские репозитории под проект «Автоматизация распределения задач пресейла»

Аналитический каталог open-source решений по подсистемам проекта. К каждому блоку: что брать, зачем, и на какой пункт ТЗ это ложится. Документ предназначен для приложения к ТЗ при передаче исполнителю.

Метрики репозитория (звёзды / коммиты / открытые issues) проверены вручную в июне 2026 там, где указаны конкретные числа. Для крупных mainstream-проектов и источников данных состояние оценивайте по ссылке — там оно очевидно.

0. Ключевая развилка: солвер в две стадии

В ТЗ заложены два этапа движка планирования, и доноры под них разные:

- **MVP — детерминированный жадный алгоритм** (топосортировка зависимостей → ранний допустимый слот с учётом capacity и дедлайнов). Фундамент здесь — **NetworkX** + собственная обвязка, а НЕ готовый солвер. Это даёт предсказуемость и быстрый запуск.
- **v2 — оптимизатор на constraint programming** (баланс загрузки, минимум перегрузов, мягкие цели с весами). Здесь донор — **PyJobShop** поверх OR-Tools CP-SAT, чтобы не писать CP-модель RCMPSP руками.

Это разделение — главный архитектурный вывод всего ресёрча: на MVP вы не тянете тяжёлый солвер, а на v2 не пишете его с нуля.

1. Ядро-солвер (ТЗ разделы 9, 18)

NetworkX — фундамент жадного алгоритма и обратного режима

[networkx/networkx](https://github.com/networkx/networkx) · BSD-3-Clause · крупный mainstream-проект, активный
<https://github.com/networkx/networkx>

Закрывает почти всю математику MVP:

- `topological_sort` — порядок обработки задач; выбрасывает `NetworkXUnfeasible`, если в графе зависимостей есть цикл (бесплатная валидация таблицы).
- `dag_longest_path` / `dag_longest_path_length` — критический путь по длительностям → обратный режим «самая ранняя реальная дата КП без дедлайна» (раздел 9).
- `descendants(node)` — все задачи ниже по графу → основа диффа «что-если» («сдвинет N задач»).
- `topological_generations` — уровни графа для пакетной обработки и для хитмапа.

ES/EF/LS/LF и слаки (запас по времени для приоритизации) — короткий прямой+обратный проход по топорядку, ~50 строк поверх NetworkX.

PyJobShop — апгрейд-путь №1 (вместо «голового» OR-Tools)

[PyJobShop/PyJobShop](#) · MIT · **107★**, **337 коммитов**, **30 open issues** — небольшая, но активно развиваемая научная библиотека (статья arXiv, февраль 2025)

<https://github.com/PyJobShop/PyJobShop>

Совпадения с T3 почти один-в-один:

- Поддерживает **RCMPSP** (мульти-проектная задача) — ваш сценарий «несколько проектов параллельно».
- **Optional task selection** — буквально lite-режим (опциональные задачи).
- **Deadlines, due dates, release dates, breaks** — дедлайны, ранние даты старта, перерывы.
- **Обобщённые precedence** (start-to-start и др.) — мягкая связь (внахлёт); обычные FS — жёсткая.
- **Multiple modes** — будущая ось сложности (множитель длительности).
- По умолчанию решатель **OR-Tools CP-SAT**; ставится `pip install pyjobshop`.

google/or-tools — эталон разделения жёстких и мягких ограничений

[google/or-tools](#), файл `examples/python/shift_scheduling_sat.py` · Apache-2.0 · крупный mainstream-проект

https://github.com/google/or-tools/blob/stable/examples/python/shift_scheduling_sat.py

Канонический шаблон под раздел 9: фиксированные назначения как жёсткие равенства, запросы как взвешенные булевы переменные в целевой функции, min/max-ограничения со стоимостью. Прямо ложится на ваши мягкие цели (баланс загрузки, минимум перегрузов, приоритет проекта, меньше переключений между проектами). Простой вход — официальный пример nurse scheduling в документации Google OR-Tools.

Референсы по эвристикам (НЕ прод-зависимости)

- [derhendrik/RCPSP_GA](#) — генетический алгоритм Хартманна для RCPSP, позиционируется как стартовая точка. https://github.com/derhendrik/RCPSP_GA
 - [mbenahmed1/ScheduleMe](#) — Simulated Annealing для RCPSP (решение = последовательность задач). <https://github.com/mbenahmed1/ScheduleMe>
 - Топик [rcpsp](#) на GitHub — сборник метаэвристик. <https://github.com/topics/rcpsp>
-

2. Telegram-бот и UI (ТЗ разделы 3, 15, 16)

aiogram 3.x — фреймворк по умолчанию

[aiogram/aiogram](#) · MIT · крупный mainstream-проект, активный
<https://github.com/aiogram/aiogram>

Полностью асинхронный; роутеры, **Finite State Machine** (ваш каркас «старт → направление → описание задачи» и мастера), magic-фильтры, middlewares (естественное место для прав доступа из раздела 16). Сильное русскоязычное комьюнити. Альтернатива — python-telegram-bot (есть встроенный [job_queue](#) под дневную сводку), но под command-first интерфейс aiogram удобнее.

aiogram-dialog — ключевая находка под UI (раздел 15)

[Tishka17/aiogram_dialog](#) · Apache-2.0 · **894★, 1 521 коммит, 41 open issue** — очень активная, есть тесты, docs (readthedocs), CI https://github.com/Tishka17/aiogram_dialog

GUI-фреймворк поверх aiogram (вдохновлён Android SDK и React). Виджеты ложатся прямо на ТЗ:

- **Calendar** → выбор дедлайна и даты возврата брифа.
- **Multiselect** → выбор исполнителей (один/двое/трое, раздел 7) и сбор lite-набора задач.
- **Counter** → переопределение capacity на день (раздел 4).
- **Checkbox / TextInput** → флаги и свободный ввод.
- Диалоги-подтверждения → ваше «Сдвину 5 задач в 3 проектах. Подтвердить?».
- **Оффлайн HTML-превью и диаграмма переходов** — UI бота можно ревьюить до написания логики.

Требует aiogram ≥ 3.14, поддерживает Python 3.10–3.14.

Скелет проекта (бот + БД + Docker в одном месте, раздел 3)

Брать один как основу:

- [wakaree/aiogram_bot_template](https://github.com/wakaree/aiogram_bot_template) · 163★, 100 коммитов, 0 open issues — наиболее полный: aiogram 3 + PostgreSQL (asyncpg) + SQLAlchemy + Alembic + FastAPI + Redis (FSM/кэш) + Fluent i18n + nginx/systemd/Docker + uv. **Рекомендую как базовый каркас.** https://github.com/wakaree/aiogram_bot_template
- [andrew000/aiogram-template](https://github.com/andrew000/aiogram-template) — aiogram + i18n + SQLAlchemy/Alembic + Postgres + Redis + Caddy + Docker + uv; в экосистеме aiogram-dialog и dishka (DI). <https://github.com/andrew000/aiogram-template>
- [vlymar1/aiogram-bot-template](https://github.com/vlymar1/aiogram-bot-template) — модульная структура handlers/services/DAO + Postgres + Redis + встроенная админ-панель со статистикой + Alembic. <https://github.com/vlymar1/aiogram-bot-template>
- [lixelv/aiogram-docker-template](https://github.com/lixelv/aiogram-docker-template) — проще; docker compose + Postgres + авто-деплой через GitHub Actions. <https://github.com/lixelv/aiogram-docker-template>
- [NotBupyc/aiogram-bot-template](https://github.com/NotBupyc/aiogram-bot-template) — Postgres/MySQL, throttling-middleware, фильтр isAdmin, ротация логов + отправка логов в TG-чат. <https://github.com/NotBupyc/aiogram-bot-template>

Планировщик сводок и напоминаний (раздел 11)

В aiogram нет встроенного шедулера — стандартный спутник **APScheduler** (MIT): cron-задача «каждый рабочий день в N часов → сводка по перегрузкам». Живой пример паттерна фоновых напоминаний: [AlertMode/task-tracker-bot](https://github.com/AlertMode/task-tracker-bot) (фоновый планировщик + асинх БД). <https://github.com/AlertMode/task-tracker-bot>

Аналоги-таск-боты — референс по грамматике команд (НЕ движок; солвера в них нет)

- [halink0803/telegram-task-manager](https://github.com/halink0803/telegram-task-manager) — проекты, задачи, /assign по reply+упоминанию, /set_status, /set_deadline, список по исполнителю. Почти готовый словарь команд. <https://github.com/halink0803/telegram-task-manager>
 - [jbinder/doforme-bot](https://github.com/jbinder/doforme-bot) — назначение задач участникам группы + статистика. <https://github.com/jbinder/doforme-bot>
 - Kaban (elcoan/kaban) — модель «канал на проект + бот», короткие команды («done», «due tomorrow», «priority high»), авто-теги #today/#overdue. Полезно для UX. <https://elcoan.github.io/kaban/>
-

3. LLM-слой: разбор команд и объяснения (ТЗ разделы 1, 2)

instructor — NL → структурный интент (Pydantic)

[instructor-ai/instructor](https://github.com/instructor-ai/instructor) · MIT · крупный mainstream-проект, активный
<https://github.com/instructor-ai/instructor>

Построен на Pydantic: типобезопасное извлечение, автоматическая валидация и **ретраи** (если модель вернёт значение, нарушающее ограничение схемы, ошибка скармливается обратно модели и запрос повторяется). Работает с Claude через `from_anthropic()`; схема интента описывается как Pydantic-модель. Это ровно ваш «LLM только парсит команды в структуру».

Нюанс на будущее: с февраля 2026 у Anthropic есть нативные structured outputs для актуальных моделей Claude — со временем можно уйти на чистый Anthropic SDK без обёртки, если не нужны мульти-провайдерность и ретраи. «Объяснения результата» — обычный промпт к Claude поверх диффа от солвера, отдельный донор не нужен.

4. Производственный календарь РФ (ТЗ раздел 5)

isdayoff — самый актуальный и авторитетный источник (живой API)

API: <https://isdayoff.ru/> · Python-обёртка [KlukvaMors/isdayoff](https://github.com/KlukvaMors/isdayoff)
<https://github.com/KlukvaMors/isdayoff>

Актуальные рабочие/праздничные/нерабочие дни из официальных источников;
РФ-календарь на 2026 уже есть (5-дневная неделя), плюс другие страны. Лучший вариант под живой пересчёт: ежегодные переносы выходных и ad-hoc указы подтягиваются корректно. Обёртка — тонкая; если она устареет, API дёргается напрямую по HTTP (тривиально).

Оффлайн-данные в JSON (если не хотите внешний вызов при каждом пересчёте)

- [d10xa/holidays-calendar](https://github.com/d10xa/holidays-calendar) — JSON с полями `preholidays` (сокращённые дни, −1 час) и `nowork` (внезапные нерабочие дни по указу). Учитывает нюансы, на которых ломается наивное «выкинуть сб/вс».
<https://github.com/d10xa/holidays-calendar>

- [iposho/holidays-calendar-ru](https://github.com/iposho/holidays-calendar-ru) — производственные календари РФ в JSON, простой API по году/месяцу/дню. <https://github.com/iposho/holidays-calendar-ru>
- [welltime/xmlcalendar](https://github.com/welltime/xmlcalendar) — апстрим многих из них, формат XML. <https://github.com/welltime/xmlcalendar>

Чисто-питоновая мультистрановая альтернатива

[workalendar](#) (PyPI) — есть класс для России, оффлайн, без сети. Но переносы РФ-праздников по постановлениям и указы отслеживает хуже живого API. Под РФ источником истины держите isdayoff/d10ха, workalendar — только если нужна оффлайн-простота.

5. Хранилище, журнал изменений, режим «что-если» (ТЗ разделы 14, 17)

База/миграции закрыты шаблонами из п.2 (Postgres + SQLAlchemy + Alembic). Отдельно — «журнал (кто, что, когда)» и «что-если на копии, источник правды не меняется».

sqlalchemy-history — журнал + откат

[corridor/sqlalchemy-history](#) (форк sqlalchemy-continuum) · **44★**, **885 коммитов**, **16 open issues** — зрелый, хорошо протестированный код, скромная популярность <https://github.com/corridor/sqlalchemy-history>

Отслеживает историю моделей; умеет откатывать данные объекта и все его связи на состояние конкретной транзакции (даже если объект был удалён); транзакции запрашиваются обычным синтаксисом SQLAlchemy. Откат к транзакции = ваше «не застываем, пока не подтвердил» + отмена replan.

bemi-sqlalchemy — аудит на уровне БД + time travel

[BemiHQ/bemi-sqlalchemy](#) <https://github.com/BemiHQ/bemi-sqlalchemy>

Change Data Capture через Write-Ahead Log PostgreSQL — ловит даже изменения через прямой SQL. Audit trails, **change reversion** (откат всех изменений в рамках запроса) и **time travel** (исторические данные без полного event sourcing). Хорошо ложится на ветку «что-если».

eventsourcing — если источником истины должен быть лог событий

PyPI [eventsourcing](#) — события как неизменяемые факты, темпоральные запросы, несколько проекций (аудит-лог vs дашборд). Идеально стыкуется с «пишет только агент»

(раздел 17), но тяжелее по входу. На MVP берите sqlalchemy-history; event sourcing — опция на будущее.

6. Визуализация загрузки: хитмап и Гант (ТЗ разделы 10, 11)

Отдельная библиотека-донор не нужна — это рендер в PNG и отправка картинкой в Telegram:

- **Гант / план:** Plotly [px.timeline](#) (задачи со start/end → диаграмма) · MIT. Для статистики — matplotlib [barh](#).
- **Хитмап занятости** (день × человек): seaborn/plotly heatmap. Отдельный цвет перегруза — граница colormap по правилу «сумма Y > X»; штриховка предварительной нагрузки (раздел 10) — matplotlib [hatch](#).

Мастер-таблица

Подсистема	Репозиторий	Лицензия	Состояние (проверено июнь 2026)	Брать / Игнорировать
Солвер — фундамент	networkx/networ kx	BSD-3	mainstream, активный	Брать (топосорт, критический путь, дифф)
Солвер v2 (CP)	PyJobShop/PyJ obShop	MIT	107★ · 337 коммитов · 30 issues	Брать (RCMPSP, optional tasks, дедлайны)
Солвер — шаблон весов	google/or-tools shift_schedu ling_sat.py	Apache-2.0	mainstream	Брать как образец жёстких/мягких целей
Солвер — референс	derhendrik/RCP SP_GA	см. репо	нишевый	Референс эвристик, не зависимость

Подсистема	Репозиторий	Лицензия	Состояние (проверено июнь 2026)	Брать / Игнорировать
Солвер — референс	mbenahmed1/S cheduleMe	см. репо	нишевый	Референс эвристик, не зависимость
TG-фреймворк	aiogram/aiogram	MIT	mainstream, активный	Брать (FSM, middlewares)
TG-UI	Tishka17/aiogra m_dialog	Apache-2.0	894★ · 1521 коммит · 41 issue	Брать (Calendar/Multis elect/Counter, подтверждения)
TG-скелет	wakaree/aiogra m_bot_template	уточнить в репо	163★ · 100 коммитов · 0 issues	Брать как каркас
TG-скелет (альт.)	andrew000/aiogr am-template	уточнить в репо	активный	Альтернатива (DI: dishka)
TG-скелет (альт.)	vlymar1/aiogram -bot-template	уточнить в репо	активный	Альтернатива (DAO + админка)
TG-скелет (альт.)	lixelv/aiogram-d ocker-template	уточнить в репо	активный	Альтернатива (простой деплой)
TG-скелет (альт.)	NotBupyc/aiogra m-bot-template	уточнить в репо	активный	Альтернатива (IsAdmin, логи)
Планировщик	APScheduler (PyPI)	MIT	mainstream	Брать (дневная сводка, напоминания)
TG-аналог	halink0803/teleg ram-task-manag er	см. репо	старый (≈2019)	Референс грамматики команд

Подсистема	Репозиторий	Лицензия	Состояние (проверено июнь 2026)	Брать / Игнорировать
TG-аналог	jbinder/doforme-bot	см. репо	нишевый	Референс назначения задач
LLM-интент	instructor-ai/instructor	MIT	mainstream, активный	Брать (NL→Pydantic, ретраи)
Календарь РФ	isdayoff.ru + KlukvaMors/isdayoff	см. репо	API живой, 2026 есть	Брать (живой источник истины)
Календарь РФ (JSON)	d10xa/holidays-calendar	см. репо	используется широко	Брать для оффлайна (preholidays/no work)
Календарь РФ (JSON)	iposho/holidays-calendar-ru	см. репо	данные по годам	Альтернатива оффлайн-данных
Журнал/откат	corridor/sqlalchemy-history	см. репо	44★ · 885 коммитов · 16 issues	Брать (журнал + откат к транзакции)
Аудит/time travel	BemiHQ/bemi-sqlalchemy	см. репо	активный	Опция (WAL CDC, реверсия)
Event sourcing	eventsourcing (PyPI)	BSD	mainstream	Опция на будущее
Визуализация	Plotly / matplotlib / seaborn	MIT	mainstream	Брать (Гант + хитмап → PNG)

Рекомендованный стартовый стек

- **Скелет:** [wakaree/aiogram_bot_template](#) (aiogram 3 + Postgres + SQLAlchemy + Alembic + Redis + Docker) — сервис «бот + солвер в одном процессе».
- **UI бота:** [aiogram-dialog](#) поверх него (Calendar / Multiselect / Counter / подтверждения + HTML-превью для дизайна интерфейса).
- **Солвер MVP:** свой жадный алгоритм на [networkx](#) (топосорт + критический путь для обратного режима + [descendants](#) для диффа «что-если»).
- **Солвер v2:** [PyJobShop](#) (RCMPSP + optional tasks под lite + дедлайны), с подглядыванием в [or-tools/shift_scheduling_sat.py](#) для весов мягких целей.
- **LLM:** [instructor](#) + Claude ([from_anthropic](#)) для NL→интент.
- **Календарь:** [KlukvaMors/isdayoff](#) (живой API, есть 2026) или JSON из [d10xa/holidays-calendar](#) для оффлайна.
- **Журнал / что-если:** [corridor/sqlalchemy-history](#) (или [bemi-sqlalchemy](#) для аудита на уровне WAL).
- **Сводки / напоминания:** APScheduler.
- **Визуализация:** Plotly [px.timeline](#) + seaborn heatmap → PNG.

Главный вывод: PyJobShop снимает самую дорогую часть будущего апгрейда (не нужно писать CP-SAT-модель RCMPSP руками), а на MVP NetworkX + жадная обвязка полностью закрывают разделы 9–12. Бот и интерфейс почти целиком закрываются связкой aiogram + aiogram-dialog + один из шаблонов.